For the problement statement, I am working on to Search for the official documentation of the German Bundestag's DIP API (Dokumentations- und Informationssystem für Parlamentsmaterialien). Find the base URL, authentication method, and a link to interactive API docs

Here's what I found on the DIP API (Bundestag/Bundesrat parliamentary documentation system):

Base URL
`https://search.dip.bundestag.de/api/v1/`

Example endpoints hang off this base, e.g. `.../aktivitaet`, `.../drucksache`, `.../drucksache-text`, `.../person`, `.../plenarprotokoll`, etc. The API can be reached via HTTP header or GET parameter.

Authentication
It uses a simple API key, not OAuth: an individual key can be requested by emailing parlamentsdokumentation@bundestag.de, and a temporary public key is also available.

The API key can be used either as an HTTP header or as a GET parameter:

- Header: `Authorization: ApiKey <your-key>`
- GET param: `?apikey=<your-key>`

Example curl:

```
curl -m 60
'https://search.dip.bundestag.de/api/v1/aktivitaet?apikey=YOUR_KE
Y'
```

Note: the publicly shared temporary key mentioned in community docs has an expiry date, so for production use you'd want to request your own via email.

Interactive API docs (Swagger UI)
The API provides documentation via SwaggerUI, available at:
`https://search.dip.bundestag.de/api/v1/swagger-ui/`

Official landing/help page
The Bundestag's own documentation entry point is at:
`https://dip.bundestag.de/über-dip/hilfe/api`

Unofficial but useful resources

- `bundesAPI/dip-bundestag-api` on GitHub — community-maintained overview with endpoint list and a Python client
- `demokratie-live/bt-dip-sdk` — a TypeScript client built against the official Swagger spec

Let me know if you'd like me to pull the Swagger spec and sketch out

example requests for specific endpoints (e.g. `drucksache`, `aktivitaet`, `plenarprotokoll`) for whatever you're building.

What are the different information that can be fetched using this API

Based on the endpoint list, the DIP API exposes several distinct types of parliamentary data, each as a "list all" endpoint plus a "get by ID" endpoint:

1. Aktivität (Activities)
`.../aktivitaet` — parliamentary activities, e.g. speeches, questions, motions tied to a person or body during proceedings.

2. Drucksache (Printed matter/documents)
`.../drucksache` — metadata for official printed documents (bills, motions, reports, etc.)
`.../drucksache-text` — the **full text** of those documents, separate from the metadata endpoint.

3. Person
`.../person` — master data on individuals (e.g. members of parliament) who appear in the parliamentary record.

4. Plenarprotokoll (Plenary session transcripts)
`.../plenarprotokoll` — metadata about plenary session protocols (stenographic records of debates).
`.../plenarprotokoll-text` — the **full text** of those session transcripts.

5. Vorgang (Procedures/legislative processes)
`.../vorgang` — metadata on legislative and parliamentary procedures — e.g. the lifecycle of a bill from introduction to passage, tracking all related documents and activities tied to one process.

Pattern across all entities

- Each has a **list endpoint** (e.g. `/drucksache`) that returns all records, filterable via query parameters (dates, election period, document type, etc. — visible in the Swagger UI).
- Each has an **ID endpoint** (e.g. `/drucksache/{id}`) returning metadata for one specific record.
- Documents and transcripts have **separate text endpoints** since full text is much larger than metadata — you fetch metadata first, then full text if needed.

In short, DIP covers: **who** (persons/MPs), **what was said/done** (activities, plenary transcripts), **what was written** (Drucksachen — bills, motions, reports), and **how it moved through parliament** (Vorgänge — the procedural thread linking documents and activities together). This mirrors the German legislative process: a *Vorgang* (procedure) is the umbrella, tied to multiple *Drucksachen* (documents) and *Aktivitäten* (activities/speeches), recorded in *Plenarprotokolle* (session transcripts), involving specific *Personen*

(MPs).

For exact filter parameters per endpoint (dates, Wahlperiode, Vorgangstyp, etc.), the Swagger UI at `https://search.dip.bundestag.de/api/v1/swagger-ui/` has the full parameter list — worth checking directly since query options vary per entity.

> can you show me the exact response structure and available query parameters for the /vorgang endpoint from the Swagger docs

There's an official PDF documentation and a raw OpenAPI spec — let me pull both for exact details.

This gave me the general query mechanics, but the actual field-by-field schema for `/vorgang` lives in the OpenAPI YAML. Let me pull that directly.

Found the raw OpenAPI YAML — let me fetch the full file to pull the exact `/vorgang` parameters and response schema.

I got the full OpenAPI spec. Here's the exact breakdown for `/vorgang`:

`GET /vorgang` — list endpoint

Query parameters (all optional, prefixed `f.` for filters):

Parameter	Type	Description
`f.datum.start`	date	Earliest document date (based on last associated document for a Vorgang)
`f.datum.end`	date	Latest document date
`f.aktualisiert.start`	date-time	Earliest last-updated timestamp
`f.aktualisiert.end`	date-time	Latest last-updated timestamp
`f.wahlperiode`	integer	Election period (Wahlperiode); repeatable for multiple periods
`f.drucksache`	integer	ID of a linked Drucksache — selects Vorgänge linked to it
`f.id`	integer	ID of the entity; repeatable to select multiple
`f.plenarprotokoll`	integer	ID of a linked Plenarprotokoll

Parameter	Type	Description
`f.dokumentart`	enum: `Drucksache`, `Plenarprotokoll`	Filters by linked document type
`f.dokumentnummer`	string	Document number of a linked Drucksache/Plenarprotokoll; repeatable
`f.drucksachetyp`	string	Type of linked Drucksache (e.g. "Antrag", "Gesetzentwurf")
`f.frage_nummer`	string	Question/list number within a Drucksache; repeatable
`f.gesta`	string	GESTA reference number (legislative tracking number); repeatable
`format`	enum: `json`, `xml`	Response format, default `json`
`cursor`	string	Pagination cursor from a previous response

Response (200): `VorgangListResponse`

```
{
  "numFound": 176,
  "cursor": "AoJwgNjC_PYCMURydWNrc2FjaGUtMjQ5MjYw",
  "documents": [ /* array of Vorgang objects, max 100 per page */
]
}
```

`GET /vorgang/{id}` — single entity

Parameters: `id` (integer, path, required), `format` (query, optional)
Response (200): a single `Vorgang` object.

`Vorgang` object schema

```
{
  "id": "84343",
  "typ": "Vorgang",
  "beratungsstand": "Abgeschlossen",
  "vorgangstyp": "Geschäftsordnung",
```

```
  "wahlperiode": 19,
  "initiative": ["Fraktion der CDU/CSU"],
  "datum": "2019-03-27",
  "aktualisiert": "2022-08-01T15:30:16+02:00",
  "titel": "Weitergeltung von Geschäftsordnungsrecht",
  "abstract": "Übernahme von Geschäftsordnungen durch den 19.
Deutschen Bundestag: ...",
  "sachgebiet": ["Bundestag"],
  "deskriptor": [ { "name": "...", "typ": "Sachbegriffe",
"fundstelle": true } ],
  "gesta": "B101",
  "zustimmungsbeduerftigkeit": ["Nein, laut Verkündung (BGBl
I)"],
  "kom": "(2017) 600 endg.",
  "ratsdok": "11018/17",
  "verkuendung": [ { "jahrgang": "2018", "seite": "409",
"ausfertigungsdatum": "2018-03-19", "verkuendungsdatum":
"2018-03-29", "fundstelle": "BGBl I 2018, 409", "pdf_url": "..."
} ],
  "inkrafttreten": [ { "datum": "2020-06-06", "erlaeuterung":
"Artikel 1 Nr. 14 ..." } ],
  "archiv": "XIX/288",
  "mitteilung": "...",
  "vorgang_verlinkung": [ { "id": "282237", "verweisung":
"Bericht", "titel": "...", "wahlperiode": 19 } ],
  "sek": "(2010) 305 endg."
}
```


Required fields: `id`, `typ`, `vorgangstyp`, `wahlperiode`, `aktualisiert`, `titel`

Key nested objects:

- `deskriptor` → thematic keywords from the ANTHES/PARTHES parliamentary thesaurus
- `verkuendung` → law promulgation details (Bundesgesetzblatt reference)
- `inkrafttreten` → dates a law (or parts of it) took effect
- `vorgang_verlinkung` → links to related Vorgänge

Note: the top-level `Vorgang` object does *not* include the full step-by-step procedural history — that's the separate `/vorgangsposition` endpoint (filterable via `f.vorgang=<id>`), which returns each procedural step (readings, referrals, votes) tied to this Vorgang.

Errors: `400` (bad request, e.g. invalid cursor), `401` (missing/invalid API key), `404` (only on `/vorgang/{id}`, entity not found).

I am completing a 3-hour Political Data Engineering case study for a Public

Affairs use case. The objective is to build a small, reproducible monitoring pipeline using the German Bundestag's DIP (Dokumentations- und Informationssystem für Parlamentsmaterialien) public API. The case study requires me to:

* query the public DIP API from a Jupyter Notebook,
* select suitable data,
* store the raw data in a SQLite database,
* create a dbt project with SQL models based on the SQLite data,
* transform the data so it can be meaningfully processed further,
* make the API query suitable for repeated retrievals,
* make the code understandable and easy to troubleshoot

I would like to decide which information to fetch first and then test the API connection. Do not provide the whole code but a skeleton for me to decide on how to work with for this task

Good instinct to nail down data selection before writing a line of pipeline code — it decides your schema, your dbt models, and how "repeated retrieval" needs to work.

1. Deciding what to fetch

For a Public Affairs monitoring use case, the entity that matters most is `Vorgang` (legislative/parliamentary procedure). It's the aggregation point — a bill, inquiry, or motion — that a Public Affairs team actually tracks over time ("where does this bill stand," "what's new in health policy this week"). Everything else (Drucksache, Plenarprotokoll, Aktivität) is a *document or event* tied to a Vorgang, useful for drill-down but noisier as a primary table.

Recommendation for a 3-hour scope:

Priority	Entity	Why
Primary	`/vorgang`	One row = one trackable political process. Has `titel`, `abstract`, `vorgangstyp`, `beratungsstand`, `sachgebiet`, `deskriptor`, `wahlperiode`, `datum`, `aktualisiert` — enough to build a meaningful "what's happening" dashboard without joining anything.
Optional stretch	`/vorgangsposition?f.vorgang={id}`	Adds the procedural timeline (readings, referrals, votes) if you have time left for a second dbt model layer.
Skip for scope	`/drucksache`, `/plenarprotokoll`, `/aktivitaet`, `/person`	Real value, but each adds a join and a new ingestion contract — not needed to demonstrate the pipeline pattern in 3 hours.

Filter parameters to fix now (these become your reproducibility contract):

- `f.wahlperiode` — pick current period (21) so the dataset is bounded and relevant.
- `f.aktualisiert.start` — this is your incremental-sync key. First run: omit or set far back. Subsequent runs: pass the last successful run's timestamp (with ~15 min overlap, per the docs) — this is what makes retrieval "repeatable" instead of a full re-pull every time.
- Optionally `f.sachgebiet` isn't a filter param in the spec, but you can filter client-side post-fetch, or use `f.drucksachetyp`/`f.gesta` if you want to scope to a specific policy area.

Decide now whether you want **one fixed policy topic** (e.g. filter deskriptor

client-side for "Klimaschutz") or **all Vorgänge of the current Wahlperiode** — this affects whether your SQLite raw table is a general mirror or a topic-scoped extract.

2. Project skeleton

```
dip-monitoring/
├── .env                        # DIP_API_KEY=...
├── notebooks/
│   └── 01_fetch_dip_data.ipynb
├── src/
│   ├── dip_client.py           # thin API wrapper + pagination
│   └── db.py                   # sqlite connection + raw table DDL
├── data/
│   └── raw.sqlite
└── dbt_project/
    ├── dbt_project.yml
    ├── profiles.yml            # points at data/raw.sqlite (dbt-sqlite adapter)
    └── models/
        ├── staging/
        │   └── stg_vorgang.sql
        └── marts/
            └── vorgang_monitoring.sql
```

3. Connection-test skeleton (`src/dip_client.py`)

```
import os
import requests

BASE_URL = "https://search.dip.bundestag.de/api/v1"

class DIPClient:
    def __init__(self, api_key: str | None = None):
        # TODO: load from env if not passed explicitly
        self.api_key = api_key or os.getenv("DIP_API_KEY")
        self.session = requests.Session()
        self.session.headers.update({"Authorization": f"ApiKey {self.api_key}"})

    def _get(self, endpoint: str, params: dict | None = None) -> dict:
        """Single GET request. Raises on non-200. No pagination logic here."""
```

```
        # TODO: url = f"{BASE_URL}/{endpoint}"
        # TODO: response = self.session.get(url, params=params,
timeout=30)
        # TODO: response.raise_for_status()
        # TODO: return response.json()
        raise NotImplementedError

    def test_connection(self) -> bool:
        """Minimal smoke test: fetch 1 Vorgang, confirm 200 +
expected keys."""
        # TODO: call self._get("vorgang", {"f.id": <some known
id>})
        # TODO: assert "documents" in result, print numFound
        raise NotImplementedError

    def fetch_vorgang_page(self, params: dict) -> dict:
        """Single page fetch — returns raw dict with
numFound/cursor/documents."""
        # TODO: return self._get("vorgang", params)
        raise NotImplementedError

    def fetch_all_vorgang(self, params: dict):
        """Generator: yields documents across all pages using
cursor pagination.
        Loop until cursor stops changing (per DIP docs)."""
        # TODO: cursor = None
        # TODO: while True:
        #           page = self.fetch_vorgang_page({**params,
"cursor": cursor} if cursor else params)
        #           yield from page["documents"]
        #           if page["cursor"] == cursor: break
        #           cursor = page["cursor"]
        raise NotImplementedError
```

4. Notebook skeleton (`01_fetch_dip_data.ipynb`)

```
# Cell 1 — setup
from dotenv import load_dotenv
load_dotenv()
from src.dip_client import DIPClient

client = DIPClient()

# Cell 2 — connection test (run this before anything else)
```

```
client.test_connection()

# Cell 3 — decide + fix your filter params here (single source of
truth)
FETCH_PARAMS = {
    "f.wahlperiode": 21,
    # "f.aktualisiert.start": ...  # set on repeated runs
}

# Cell 4 — pull + inspect a small sample before full run
sample = client.fetch_vorgang_page(FETCH_PARAMS)
# TODO: inspect sample["documents"][0] to confirm schema
assumptions

# Cell 5 — full pull → SQLite (later step)
```

Before you go further: do you want to scope to a single policy topic (deskriptor/Sachgebiet filter) or pull the full current Wahlperiode? That decision changes what `FETCH_PARAMS` and your raw SQLite schema look like, so worth locking in before you build `test_connection()` out fully.

> i missed to mention the specific policy topic i am working on, concerning the topic of pension policy in Germany to create an early warning system

Good — that reframes a few decisions. Pension policy ("Rente") has no dedicated server-side filter in the DIP API (no `f.sachgebiet` or `f.deskriptor` param exists), so the topic scoping has to happen **client-side, after fetching**, against fields already in the `Vorgang` object:

- `sachgebiet` (array of policy areas, e.g. "Rentenversicherung")
- `deskriptor` (array of thesaurus keywords — more granular, e.g. "Altersrente", "Rentenanpassung", "Grundrente")
- `titel` / `abstract` (free text, useful as a fallback regex match)

This changes your pipeline shape a bit: you fetch broadly (by `f.wahlperiode`), then filter, rather than fetching narrowly.

Updated decisions

1. Keyword list — fix this now, it's your topic contract

```
PENSION_KEYWORDS = [
    "Rente", "Renten", "Altersrente", "Rentenversicherung",
    "Grundrente", "Rentenanpassung", "Alterssicherung",
    "Betriebsrente", "Erwerbsminderungsrente",
]
```

Decide: match against `sachgebiet` + `deskriptor` only (cleaner, controlled vocabulary) or also fall back to `titel`/`abstract` regex (higher recall, more noise). For an early-warning system, controlled-vocabulary-first with a text fallback flagged separately is a defensible design choice — worth stating explicitly in your write-up either way.

2. "Early warning" reframes what you're storing
An early warning system cares about two things a plain monitoring pipeline doesn't need to distinguish as sharply:

- **New** Vorgang matching pension criteria (first time seen)
- **Updated** Vorgang (a pension-related process moved — `beratungsstand` changed, new `vorgangsposition`, etc.)

That means your SQLite layer needs a way to diff against the previous run, not just append raw data. Two common patterns:

- Store every fetched snapshot with a `fetched_at` timestamp and let dbt diff `aktualisiert` across runs (simpler, more storage).
- Keep a small `pipeline_state` table with `last_run_timestamp`, feed that into `f.aktualisiert.start` on the next fetch, and only ever insert what's new (leaner, but the "diff" logic moves into the fetch step).

For a 3-hour scope, the first pattern is easier to make correct and explain; the second is more realistic for production. Worth deciding which one you want to demonstrate.

Updated skeleton

```
dip-monitoring/
├── src/
│   ├── dip_client.py         # unchanged from before
│   ├── filters.py            # NEW: pension keyword matching
logic
│   └── db.py
├── data/raw.sqlite
└── dbt_project/
    └── models/
        ├── staging/
        │   └── stg_vorgang.sql
        └── marts/
            ├── pension_vorgang_current.sql   # latest state per
Vorgang
            └── pension_alerts.sql            # NEW/UPDATED since
last run
```

`src/filters.py`

```
def is_pension_related(vorgang: dict) -> bool:
    """Check sachgebiet + deskriptor against PENSION_KEYWORDS.
    Return True/False. Keep this pure + testable — no I/O."""
    # TODO: check vorgang.get("sachgebiet", [])
    # TODO: check [d["name"] for d in vorgang.get("deskriptor",
[])]
    raise NotImplementedError
```

Notebook cell — decide fetch scope

```
FETCH_PARAMS = {
    "f.wahlperiode": 21,
    # "f.aktualisiert.start": <last_run_timestamp>,  # set
programmatically on repeat runs
}

# Fetch broad, filter narrow — because DIP has no topic-level
query param
all_vorgaenge = client.fetch_all_vorgang(FETCH_PARAMS)
pension_vorgaenge = [v for v in all_vorgaenge if
is_pension_related(v)]
```

One thing worth deciding before you build further: do you want the raw SQLite table to hold **all** fetched Vorgänge (with a `matches_pension_topic` boolean column, so you can adjust the keyword list later without re-fetching), or only the already-filtered subset (smaller, but a keyword-list change means a full re-pull)? For a monitoring/early-warning use case where the topic definition might get refined, I'd lean toward storing broad and filtering in dbt — but it's your call on scope given the 3-hour budget.

> API key works well and i ran a smoke test which provide below output {'id': '337670', 'beratungsstand': 'Beantwortet', 'vorgangstyp': 'Schriftliche Frage', 'typ': 'Vorgang', 'wahlperiode': 21, 'aktualisiert': '2026-08-05T14:55:39+02:00', 'deskriptor': [{'fundstelle': False, 'name': 'Rente', 'typ': 'Sachbegriffe'}, {'fundstelle': False, 'name': 'Renteneintrittsalter', 'typ': 'Sachbegriffe'}, {'fundstelle': True, 'name': 'Rentenreform', 'typ': 'Sachbegriffe'}], 'titel': 'Abschaffung der abschlagsfreien Rente', 'datum': '2026-07-17'}

That's a clean smoke test — and this specific record is genuinely useful as a first test fixture, since it's a real pension-related `Vorgang` with a nice edge case built in.

A few things worth noticing in this response:

- **`fundstelle` is doing real work here.** Three deskriptoren matched:

`Rente`, `Renteneintrittsalter`, `Rentenreform` — but only `Rentenreform` has `fundstelle: True`. Recall from the schema: `fundstelle` marks the descriptor as *central* to the Vorgang, not just tangentially mentioned. For an early-warning system, this is a natural relevance signal you get for free — you could weight or flag matches differently depending on whether they hit a `fundstelle: True` descriptor vs. a peripheral one. Worth deciding now whether `is_pension_related()` returns a plain bool, or something richer like a match reason/confidence tier.

- **No `sachgebiet` field in this response.** The schema lists it as optional, and this record doesn't have it — so if your filter logic checks `sachgebiet` first and falls back to `deskriptor`, this record confirms you can't rely on `sachgebiet` alone. Good real-world validation of the "deskriptor-first" approach.

- **No `abstract` field either** — also optional per the schema, absent here. If your dbt staging model expects `abstract`, decide now whether missing values become `NULL` or an empty string, since `Schriftliche Frage` (written question) records may commonly lack one compared to bills.

- **`vorgangstyp: 'Schriftliche Frage'`** is itself informative for an early-warning system — written parliamentary questions are often the earliest signal that an issue is heating up, before it becomes a bill. You might want `vorgangstyp` as a field you surface/prioritize in your alerts model rather than just carry through as metadata.

Suggested next step: use this exact record as a unit-test fixture for `filters.py` before you touch the fetch loop —

```
# tests/test_filters.py
SAMPLE_VORGANG = {
    "id": "337670",
    "deskriptor": [
        {"fundstelle": False, "name": "Rente", "typ":
"Sachbegriffe"},
        {"fundstelle": False, "name": "Renteneintrittsalter",
"typ": "Sachbegriffe"},
        {"fundstelle": True, "name": "Rentenreform", "typ":
"Sachbegriffe"},
    ],
    "titel": "Abschaffung der abschlagsfreien Rente",
    # no sachgebiet, no abstract — matches real API behavior
}
```

```
def test_is_pension_related_matches_via_deskriptor():
    assert is_pension_related(SAMPLE_VORGANG) is True
```

This forces you to write `filters.py` against real field shapes instead of the OpenAPI schema's idealized example, which is exactly where these pipelines tend to break later.

Once that's locked in, next decision point: do you want `fetch_all_vorgang()` to pull the full `f.wahlperiode=21` set and filter after (my earlier recommendation), or are you reconsidering given how specific `deskriptor` matching already looks in practice?

i would like to use following search terms QUERIES = [
{"f.titel": "Rente"},
{"f.titel": "Rentenversicherung"},
{"f.titel": "Altersvorsorge"},
{"f.titel": "Grundrente"},
{"f.titel": "Rentenreform"},

Worth pausing on this — `f.titel` isn't a documented parameter for `/vorgang`. Looking back at the OpenAPI spec parameters we pulled for that endpoint:

```
f.datum.start, f.datum.end, f.aktualisiert.start,
f.aktualisiert.end,
f.wahlperiode, f.drucksache, f.id, f.plenarprotokoll,
f.dokumentart,
f.dokumentnummer, f.drucksachetyp, f.frage_nummer, f.gesta,
format, cursor
```

No `f.titel`, and no general full-text search parameter (`q` or similar) either. The API's design (per the docs) is filter-by-linked-entity or filter-by-metadata, not free-text search over titles.

What likely happens if you send `f.titel=Rente` anyway: the API will probably either silently ignore the unknown parameter and return the unfiltered full list, or reject it — worth testing once to see, but don't rely on either behavior for production logic.

Recommendation: stick with the fetch-broad, filter-client-side approach we set up earlier via `is_pension_related()`. Your five terms map cleanly onto that already:

```
PENSION_KEYWORDS = [
    "Rente",
    "Rentenversicherung",
    "Altersvorsorge",
    "Grundrente",
    "Rentenreform",
```

```
]
```

The only decision left is **where** to match them — and your smoke-test record already told you the answer: match against `deskriptor[].name` (and optionally `titel` as a fallback for recall), not `sachgebiet`, since that field wasn't even present.

```
def is_pension_related(vorgang: dict, keywords: list[str] =
PENSION_KEYWORDS) -> bool:
    descriptor_names = [d["name"] for d in
vorgang.get("deskriptor", [])]
    if any(k in descriptor_names for k in keywords):
        return True
    # fallback: substring match on titel, catches cases with
no/partial deskriptor
    titel = vorgang.get("titel", "")
    return any(k.lower() in titel.lower() for k in keywords)
```

Quick empirical check before you commit to this — worth running once in your notebook to confirm the unknown-parameter behavior rather than assume it:

```
# does the API reject, ignore, or (surprisingly) honor an
undocumented param?
test = client.fetch_vorgang_page({"f.wahlperiode": 21, "f.titel":
"Rente"})
print(test["numFound"])  # compare against unfiltered numFound
for f.wahlperiode=21 alone
```

If the numbers are identical, it's silently ignoring the param — good confirmation to lock in the client-side filtering path and move on.

i have worked on the fetch and ran the code pasting here from my notebook, the output

```
#search terms
QUERIES = [
{"f.titel": "Rente"},
{"f.titel": "Rentenversicherung"},
{"f.titel": "Altersvorsorge"},
{"f.titel": "Grundrente"},
{"f.titel": "Rentenreform"},
# Restrict to the current electoral term to keep the first pull small;
# remove this filter for a full historical backfill.
# {"f.wahlperiode": 21},
]


# Each query is run independently and its own success/failure is logged
# and recorded in fetch_log
results = {}
for params in QUERIES:
term = params["f.titel"]
```

```
            try:
                count = fetch_and_store(client, conn, resource="vorgang", params=params)
                results[term] = f"OK ({count} new rows)"
            except Exception as exc:
                results[term] = f"FAILED: {type(exc).__name__}: {exc}"

        print("Per-query results:")
        for term, outcome in results.items():
            print(f"  {term!r}: {outcome}")

        failed = [t for t, o in results.items() if o.startswith("FAILED")]
        if failed:
            print(f"\n{len(failed)} of {len(QUERIES)} queries failed -- check fetch_log for details before trusting downstream dbt output.")
```

Good news first: `f.titel` does work — it's an undocumented but apparently real full-text-ish filter on the title field (the varying `numFound` per term across 692/949/163/37/25 confirms it's not being silently ignored). Worth correcting my earlier caution: empirically this API accepts more than what's in the published OpenAPI spec. Good instinct to test it rather than trust the spec blindly.

That said, this result surfaces two things worth fixing before you trust the SQLite table downstream:

1. Overlap between queries → likely duplicate rows

A `Vorgang` titled e.g. "Reform der gesetzlichen Rentenversicherung" would match both `"Rente"` and `"Rentenversicherung"` and get fetched twice, once per query loop iteration. Whether that's a problem depends entirely on how `fetch_and_store` writes to SQLite:

- If your `vorgang` table has a `UNIQUE` (or `PRIMARY KEY`) constraint on `id` and you're using `INSERT OR IGNORE` / `INSERT OR REPLACE`, duplicates across queries collapse automatically — the "new rows" counts you're printing are just per-query fetch counts, not necessarily net-new distinct rows in the table.
- If it's a plain `INSERT`, you now have real duplicate `id`s in the raw table, and 1,866 raw rows overstates your actual distinct Vorgänge.

Worth checking right now:

```
cur = conn.execute("SELECT COUNT(*) FROM vorgang")
print("total rows:", cur.fetchone()[0])

cur = conn.execute("SELECT COUNT(DISTINCT id) FROM vorgang")
print("distinct ids:", cur.fetchone()[0])
```

If those two numbers differ, fix the insert logic (or the table schema) before moving to dbt — dedup is much easier to guarantee correct at the SQLite write layer than to redo later in a staging model.

2. `f.wahlperiode` is commented out — this was a full historical

backfill, not scoped to current activity

Your comment says this intentionally, which is a reasonable call for a first pull, but it means the ~1,866 rows span the entire history DIP has on these terms (Vorgänge date back to the 8th Wahlperiode / 1976), not just the current term. For an *early-warning* system specifically, that's a lot of dead weight — most of this data is procedurally closed and not something anyone needs to be "warned" about. Two paths:

- Keep the historical pull as a one-time backfill/reference table, but build your alerting logic on a `WHERE wahlperiode = 21` (or `beratungsstand != 'Abgeschlossen'`) filtered view in dbt.
- Or re-run with `f.wahlperiode: 21` uncommented now, before building dbt models, so the raw table itself stays lean and matches your case study's stated purpose.

Given your 3-hour budget, I'd lean toward re-running scoped to wahlperiode 21 now rather than filtering 1,866 historical rows down in dbt — less data to reason about, and "current term only" is an easy, defensible design choice to state in your write-up.

Which do you want to do — check/fix the dedup logic first, or re-scope the fetch to the current Wahlperiode?

i have already test this and would like to pull all the data first to keep it raw format. i ran a sanity check and have below output
<style scoped>
.dataframe tbody tr th:only-of-type {
vertical-align: middle;
}
.dataframe tbody tr th {
vertical-align: top;
}
.dataframe thead th {
text-align: right;
}
</style>
<table border="1" class="dataframe">
<thead>
<tr style="text-align: right;">
<th></th>
<th>id</th>
<th>resource</th>
<th>params_json</th>
<th>requested_at</th>

<th>item_count</th>
<th>error</th>
</tr>
</thead>
<tbody>
<tr>
<th>0</th>
<td>1</td>
<td>vorgang</td>
<td>{"f.titel": "Rente"}</td>
<td>2026-08-23T21:55:50.850208+00:00</td>
<td>692</td>
<td>None</td>
</tr>
<tr>
<th>1</th>
<td>2</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenversicherung"}</td>
<td>2026-08-23T21:55:51.619391+00:00</td>
<td>949</td>
<td>None</td>
</tr>
<tr>
<th>2</th>
<td>3</td>
<td>vorgang</td>
<td>{"f.titel": "Altersvorsorge"}</td>
<td>2026-08-23T21:55:51.782127+00:00</td>
<td>163</td>
<td>None</td>
</tr>
<tr>
<th>3</th>
<td>4</td>
<td>vorgang</td>
<td>{"f.titel": "Grundrente"}</td>
<td>2026-08-23T21:55:51.879346+00:00</td>
<td>37</td>
<td>None</td>
</tr>
<tr>

<th>4</th>
<td>5</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenreform"}</td>
<td>2026-08-23T21:55:51.977118+00:00</td>
<td>25</td>
<td>None</td>
</tr>
<tr>
<th>5</th>
<td>6</td>
<td>vorgang</td>
<td>{"f.titel": "Rente"}</td>
<td>2026-08-24T07:42:29.703088+00:00</td>
<td>692</td>
<td>None</td>
</tr>
<tr>
<th>6</th>
<td>7</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenversicherung"}</td>
<td>2026-08-24T07:42:31.531360+00:00</td>
<td>949</td>
<td>None</td>
</tr>
<tr>
<th>7</th>
<td>8</td>
<td>vorgang</td>
<td>{"f.titel": "Altersvorsorge"}</td>
<td>2026-08-24T07:42:32.044957+00:00</td>
<td>163</td>
<td>None</td>
</tr>
<tr>
<th>8</th>
<td>9</td>
<td>vorgang</td>
<td>{"f.titel": "Grundrente"}</td>
<td>2026-08-24T07:42:32.213652+00:00</td>
<td>37</td>

<td>None</td>
</tr>
<tr>
<th>9</th>
<td>10</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenreform"}</td>
<td>2026-08-24T07:42:32.451165+00:00</td>
<td>25</td>
<td>None</td>
</tr>
<tr>
<th>10</th>
<td>11</td>
<td>vorgang</td>
<td>{"f.titel": "Rente"}</td>
<td>2026-08-25T12:54:52.410089+00:00</td>
<td>692</td>
<td>None</td>
</tr>
<tr>
<th>11</th>
<td>12</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenversicherung"}</td>
<td>2026-08-25T12:55:14.505053+00:00</td>
<td>949</td>
<td>None</td>
</tr>
<tr>
<th>12</th>
<td>13</td>
<td>vorgang</td>
<td>{"f.titel": "Altersvorsorge"}</td>
<td>2026-08-25T12:55:16.648996+00:00</td>
<td>163</td>
<td>None</td>
</tr>
<tr>
<th>13</th>
<td>14</td>
<td>vorgang</td>

<td>{"f.titel": "Grundrente"}</td>
<td>2026-08-25T12:55:17.102047+00:00</td>
<td>37</td>
<td>None</td>
</tr>
<tr>
<th>14</th>
<td>15</td>
<td>vorgang</td>
<td>{"f.titel": "Rentenreform"}</td>
<td>2026-08-25T12:55:17.560759+00:00</td>
<td>25</td>
<td>None</td>
</tr>
</tbody>
</table>

This `fetch_log` table is a nice design element — logging every API call with params, timestamp, and item_count as its own metadata table is exactly the kind of thing that makes a pipeline auditable/troubleshootable, which is one of your stated requirements. Keep it.

What it confirms: the same 5 queries return identical `item_count`s across three separate runs (Aug 23, 24, 25) — 692 / 949 / 163 / 37 / 25 every time. That's a good sign for reproducibility at the API level: the DIP responses are stable for these terms over this window, and your fetch logic is deterministic.

What it does* not *confirm: whether your actual `vorgang` data table (not this log table) deduplicates across those three runs. `fetch_log` just records that the API call happened and how many items came back each time — it says nothing about what happened when those items were written to the `vorgang` table. You ran the identical set of 5 queries three times now, so this is actually the exact test case to check:

```
cur = conn.execute("SELECT COUNT(*) FROM vorgang")
print("total rows:", cur.fetchone()[0])

cur = conn.execute("SELECT COUNT(DISTINCT id) FROM vorgang")
print("distinct ids:", cur.fetchone()[0])
```

Three possible outcomes, and each tells you something different about what to do next:

- **`total = distinct` and both ≈ some number less than 1,866×3** → your writes are already idempotent (upsert-style), duplicates across runs *and* across overlapping search terms are being collapsed correctly.

You're in good shape to move to dbt as-is.

- **`total = distinct` but `total ≈ 1866`** → your writes dedupe within a single run's cross-query overlap but you're only looking at the last run's data (e.g. table was dropped/recreated each run rather than appended to) — worth confirming that's intentional given you said you want raw format preserved.
- **`total > distinct`** → plain appends, duplicate `id`s are sitting in the raw table. Since you explicitly want to *keep the data raw*, this isn't necessarily wrong — but if so, the burden of deduplication just moves downstream. In that case, your dbt staging model (`stg_vorgang.sql`) needs a `ROW_NUMBER() OVER (PARTITION BY id ORDER BY fetched_at DESC)`-style dedup as its first transformation, and you should note in your write-up that the raw layer is append-only/log-style by design while staging enforces "one row per Vorgang."

Given you want raw format preserved, that third scenario is actually a defensible design — raw = full fetch history, staging = deduplicated current state. But you need to know which of the three you're actually in before writing the dbt model, since it changes what `stg_vorgang.sql`'s first CTE needs to do.

> i would like first just fetch one record from db and fetch vorgangsposition for this Vorgang to find the Drucksache link

Good move — pull one real record and trace it through to `Drucksache` before building the general pipeline logic. This is exactly the kind of manual "walk one record end-to-end" check that catches schema-shape surprises before they hit dbt.

Step 1 — pull one row from your `vorgang` table

```
cur = conn.execute("SELECT id, titel, vorgangstyp FROM vorgang
LIMIT 1")
row = cur.fetchone()
print(row)

vorgang_id = row[0]
```

Grab an actual `id` — don't hardcode one yet, let the DB tell you what's there. If your table stores the full raw JSON in a column too (worth checking — depends on how `fetch_and_store` wrote it), inspect that as well so you know exactly what fields survived the write.

Step 2 — fetch `Vorgangsposition` for that Vorgang

Recall from the schema: `/vorgangsposition` takes `f.vorgang` as a filter (integer ID of the linked Vorgang), and each `Vorgangsposition` object carries a `fundstelle` object — that's where the `Drucksache` link actually lives, via

`fundstelle.dokumentart` and `fundstelle.id`.

```
positions = client.fetch_vorgang_page  # not this — need a
Vorgangsposition fetch method
# TODO: does your DIPClient have a generic fetch method, or only
vorgang-specific ones?
# if generic:
positions = list(client.fetch_all(resource="vorgangsposition",
params={"f.vorgang": vorgang_id}))

print(f"{len(positions)} Vorgangsposition(en) found for Vorgang
{vorgang_id}")
for p in positions:
    print(p["vorgangsposition"], "->",
p["fundstelle"]["dokumentart"], p["fundstelle"]["id"])
```

Worth checking now, before writing this: does your `DIPClient` class have a generic `resource` parameter on its fetch methods, or did you write `fetch_all_vorgang` as a `vorgang`-specific method only? If it's the latter, this is the natural point to generalize it — e.g. `fetch_all(resource: str, params: dict)` — since you'll want the same pagination/cursor logic for `vorgangsposition`, and later possibly `drucksache`, without duplicating the loop.

Step 3 — isolate the Drucksache references

```
drucksache_refs = [
    p["fundstelle"]["id"]
    for p in positions
    if p["fundstelle"]["dokumentart"] == "Drucksache"
]
print("Linked Drucksache IDs:", drucksache_refs)
```

A couple of things to watch for once you see real output:

- Not every `Vorgangsposition` will have `dokumentart == "Drucksache"` — some will be `"Plenarprotokoll"` (debate references). Your early-warning use case probably cares about both eventually, but for this trace, filtering to `Drucksache` is the right first step.
- The `gang` boolean on `Vorgangsposition` marks steps "of particular importance to the progress of deliberation" — worth checking whether the Drucksache-linked steps you find are `gang: true` or not, since that's a natural signal for which document links matter most in an alert.

Once you've got one real `Drucksache` ID out of this, do you want to fetch that Drucksache's metadata next (via `/drucksache/{id}`) to see what it looks like, before deciding whether it's worth a dedicated table in your dbt models?

it works as below output Real response structure + title field (from fetched data) ===
Top-level keys present in a real Vorgang record:
['abstract', 'aktualisiert', 'beratungsstand', 'datum', 'deskriptor', 'id', 'initiative', 'sachgebiet', 'titel', 'typ', 'vorgangstyp', 'wahlperiode']
Title field ('titel'): Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113 Titel 656 03 - Beteiligung des Bundes in knappschaftlicher Rentenversicherung - (G-SIG: 14000059)
Full sample record:


```
{
"id": "100154",
"abstract": "Gewährung eines Zuschusses bis zur Höhe von 279,7 Mio. DM ",
"beratungsstand": "Abgeschlossen - Ergebnis siehe Vorgangsablauf",
"vorgangstyp": "Über- und außerplanmäßige Haushaltsausgaben",
"sachgebiet": [
"Soziale Sicherung",
"Öffentliche Finanzen, Steuern und Abgaben"
],
"typ": "Vorgang",
"wahlperiode": 14,
"initiative": [
"Bundesministerium der Finanzen"
],
"aktualisiert": "2022-07-26T19:57:25+02:00",
"deskriptor": [
{
"fundstelle": false,
"name": "Haushalt",
"typ": "Freier Deskriptor"
},
{
"fundstelle": false,
"name": "Knappschaftliche Rentenversicherung",
"typ": "Sachbegriffe"
},
{
"fundstelle": false,
"name": "Rentenversicherung",
"typ": "Freier Deskriptor"
},
{
"fundstelle": true,
```

"name": "Über- und außerplanmäßige Haushaltsausgaben Haushaltsjahr 1998",
"typ": "Freier Deskriptor"
}
],
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113 Titel 656 03 - Beteiligung des Bundes in knappschaftlicher Rentenversicherung - (G-SIG: 14000059)",
"datum": "1999-01-28"
}
Vorgang -> Drucksache link (via /vorgangsposition for vorgang id=100154) ===
2026-08-25 15:06:01,428 INFO dip_client: [eaa4db30] GET vorgangsposition params={'f.vorgang': '100154', 'format': 'json'} -> 200 (500 ms)
2026-08-25 15:06:01,428 INFO dip_client: resource=vorgangsposition page=1 items=4 numFound=4
2026-08-25 15:06:01,609 INFO dip_client: [31b2af7c] GET vorgangsposition params={'f.vorgang': '100154', 'format': 'json', 'cursor': 'AoJcuQI3Vm9yZ2FuZ3Nwb3NpdGlvbi0yMDk2MDU='} -> 200 (188 ms)
2026-08-25 15:06:01,611 INFO dip_client: [31b2af7c] Zero results for this query (numFound=4) -- not an error
2026-08-25 15:06:01,611 INFO dip_client: resource=vorgangsposition page=2 items=0 numFound=4
Found 4 vorgangsposition record(s)
Keys in a vorgangsposition record: ['aktivitaet_anzahl', 'aktualisiert', 'datum', 'dokumentart', 'fortsetzung', 'fundstelle', 'gang', 'id', 'nachtrag', 'titel', 'typ', 'ueberweisung', 'urheber', 'vorgang_id', 'vorgangsposition', 'vorgangstyp', 'zuordnung']
Full sample vorgangsposition record:
{
"id": "209602",
"vorgangsposition": "Unterrichtung",
"zuordnung": "BR",
"gang": true,
"fortsetzung": false,
"nachtrag": false,
"vorgangstyp": "Über- und außerplanmäßige Haushaltsausgaben",
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113 Titel 656 03 - Beteiligung des Bundes in knappschaftlicher Rentenversicherung - (G-SIG: 14000059)",
"typ": "Vorgangsposition",
"aktivitaet_anzahl": 0,

```
"dokumentart": "Drucksache",
"aktualisiert": "2022-07-26T19:57:15+02:00",
"urheber": [
{
"einbringer": false,
"bezeichnung": "BMF",
"titel": "Bundesministerium der Finanzen"
}
],
"ueberweisung": [
{
"ausschuss": "Finanzausschuss",
"ausschuss_kuerzel": "Fz",
"federfuehrung": true
}
],
"fundstelle": {
"id": "82891",
"dokumentnummer": "908/98",
"datum": "1998-11-13",
"dokumentart": "Drucksache",
"drucksachetyp": "Unterrichtung",
"herausgeber": "BR",
"urheber": [
"Bundesministerium der Finanzen"
],
"pdf_url": "https://dserver.bundestag.de/brd/1998/D908+98.pdf"
},
"datum": "1998-11-13",
"vorgang_id": "100154"
}
```


This trace worked end-to-end and surfaced something important: **this particular record is a false positive for your use case.**

Look at what actually matched: `id=100154` is a 1999 budget matter — a supplementary appropriation ("Über- und außerplanmäßige Haushaltsausgaben") for the Bundesamt's contribution to *knappschaftliche* Rentenversicherung (the miners'/coal-sector pension scheme). It matched your keyword search on `"Rentenversicherung"` purely because that word appears in the title/deskriptor — but this isn't a pension *policy* development in any meaningful sense for an early-warning system. It's a budget execution notice from 27 years ago, already closed (`beratungsstand: "Abgeschlossen"`), from Wahlperiode 14.

This is exactly the kind of signal you want to catch now, before dbt, not after:

- **`f.titel` substring matching produces false positives.** "Rentenversicherung" matches both genuine pension-reform Vorgänge and incidental budget line items that happen to mention it. Worth deciding whether to accept this noise (label it, don't filter it — let the analyst see it) or tighten the filter (e.g. require `deskriptor[].fundstelle == true` AND the descriptor name in your keyword list, which is a much stronger relevance signal than title substring match).

- **This confirms you need the `f.wahlperiode` decision you deferred.** A `Vorgang` from 1999 that's already `Abgeschlossen` has zero early-warning value — it's historical noise. This is good concrete evidence for scoping your *alerting* dbt model to current/recent Wahlperioden and open `beratungsstand`, even if the raw table stays full-history.

On the actual Vorgang → Drucksache link mechanics — this part worked cleanly:

- `vorgangsposition.dokumentart == "Drucksache"` ✓
- `vorgangsposition.fundstelle.id` = `"82891"` → this is the **Drucksache ID** you'd use to fetch `/drucksache/82891`
- `vorgangsposition.fundstelle.dokumentnummer` = `"908/98"`, `pdf_url` already included — you may not even need a separate `/drucksache/{id}` call if `fundstelle` already carries what you need (dokumentnummer, datum, drucksachetyp, herausgeber, pdf_url are all there).
- `gang: true` — this step is flagged as procedurally significant, consistent with what we discussed earlier.

Decision point before building the dbt models: does `vorgangsposition.fundstelle` give you enough Drucksache info already (dokumentnummer, pdf_url, drucksachetyp, datum), or do you actually need the full `/drucksache/{id}` record (author list, `vorgangsbezug`, etc.)? If `fundstelle` is sufficient, you can skip a whole extra fetch step and entity table — `vorgangsposition` becomes your join table between `vorgang` and Drucksache metadata, with no separate `drucksache` raw table needed at all. That would meaningfully simplify your dbt project (one fewer staging model) and is worth deciding now given your time budget.

What's your read — does `fundstelle` cover what you need, or do you want the fuller Drucksache record too?

i would like to verify the Drucksache -> Drucksache-Text relationship before building on it

Good instinct to check this before assuming it — and there's a specific reason to be skeptical with *this exact record*. Recall from your earlier search results: Bundesrat Drucksachen only have full-text search available from 2003 onward, while metadata goes back to 1984. Your traced Drucksache (`id=82891`, `herausgeber: BR`, `datum: 1998-11-13`) predates that — so it's a good real-world test case for what happens when the full text simply isn't available.

What the schema says the relationship should be

From the OpenAPI spec: `DrucksacheText` is defined as `allOf: [Drucksache, DokumentTextBase]` — meaning `/drucksache-text/{id}` should return every field `/drucksache/{id}` does, plus one additional field: `text` (the full document body). Same `id`, same endpoint pattern, just a superset schema. In theory, `/drucksache-text/82891` and `/drucksache/82891` should share the same `id` and overlapping fields.

Test it directly

```
drucksache_id = "82891"

# Metadata-only endpoint
meta = client._get(f"drucksache/{drucksache_id}")
print("=== /drucksache/{id} ===")
print(json.dumps(meta, indent=2, ensure_ascii=False)[:1000])

# Text endpoint
try:
    full = client._get(f"drucksache-text/{drucksache_id}")
    print("\n=== /drucksache-text/{id} ===")
    print("Keys:", list(full.keys()))
    print("Has 'text' field:", "text" in full)
    print("Text length:", len(full.get("text", "")))
except Exception as exc:
    print(f"/drucksache-text/{drucksache_id} failed:
{type(exc).__name__}: {exc}")
```

What to check in the output:

- **Does `/drucksache-text/82891` return 200, or 404/empty?** Given the 1998 BR date, this may genuinely have no indexed full text — that's not a bug in your pipeline, it's a documented coverage gap. Worth confirming rather than assuming your fetch logic is broken if it comes back empty.
- **If it does return data, does `meta["id"] == full["id"]`** and do the shared fields (`titel`, `dokumentnummer`, `datum`, etc.) match between the two calls? This confirms the "same entity, richer payload" relationship the schema implies rather than two independently-drifting resources.

- **If empty/404 on this record, retest with a recent BT Drucksache instead** — e.g. pull an `id` from your `vorgang`/`vorgangsposition` trace where `wahlperiode` is 21 and `herausgeber` is `BT`, since Bundestag full-text goes back to 1949 per the docs, so a current-term BT record is a cleaner positive test case.

Worth deciding once you see the result: do you actually need `/drucksache-text` at all for the early-warning use case, or is `/drucksache` metadata (title, type, date, author, PDF link) enough to alert on — with full text only fetched on-demand when an analyst drills into a specific flagged item? Given full-text responses are capped at 10 per page (vs. 100 for metadata) and are likely large payloads, pulling text speculatively for every matched Vorgang may not be worth the storage/time cost in a 3-hour pipeline.

this retuns below 2026-08-25 15:10:04,931 INFO dip_client: [e27198f8] GET drucksache params={'f.id': '82891', 'format': 'json'} -> 200 (469 ms)
2026-08-25 15:10:04,933 INFO dip_client: resource=drucksache page=1 items=1 numFound=1
2026-08-25 15:10:05,116 INFO dip_client: [582d763a] GET drucksache-text params={'f.id': '82891', 'format': 'json'} -> 200 (188 ms)
=== /drucksache response ===


```
{
"id": "82891",
"drucksachetyp": "Unterrichtung",
"dokumentart": "Drucksache",
"autoren_anzahl": 0,
"typ": "Dokument",
"vorgangsbezug_anzahl": 1,
"dokumentnummer": "908/98",
"wahlperiode": 14,
"herausgeber": "BR",
"pdf_hash": "087a7c94c85bb5de691b2049a9a67dc6",
"aktualisiert": "2022-07-26T19:57:15+02:00",
"vorgangsbezug": [
{
"id": "100154",
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113 Titel 656 03 - Beteiligung des Bundes in knappschaftlicher Rentenversicherung - (G-SIG: 14000059)",
"vorgangstyp": "Über- und außerplanmäßige Haushaltsausgaben"
}
],
"urheber": [
```

{
"einbringer": false,
"bezeichnung": "BMF",
"titel": "Bundesministerium der Finanzen"
}
],
"fundstelle": {
"pdf_url": "https://dserver.bundestag.de/brd/1998/D908+98.pdf",
"id": "82891",
"dokumentnummer": "908/98",
"datum": "1998-11-13",
"dokumentart": "Drucksache",
"drucksachetyp": "Unterrichtung",
"herausgeber": "BR",
"urheber": [
"Bundesministerium der Finanzen"
]
},
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113 Titel 656 03 - Beteiligung des Bundes in knappschaftlicher Rentenversicherung -",
"datum": "1998-11-13"
}
2026-08-25 15:10:05,116 INFO dip_client: resource=drucksache-text page=1 items=1 numFound=1
=== /drucksache-text response (same id?) ===
{
"id": "82891",
"drucksachetyp": "Unterrichtung",
"dokumentart": "Drucksache",
"autoren_anzahl": 0,
"typ": "Dokument",
"vorgangsbezug_anzahl": 1,
"dokumentnummer": "908/98",
"wahlperiode": 14,
"herausgeber": "BR",
"pdf_hash": "087a7c94c85bb5de691b2049a9a67dc6",
"aktualisiert": "2022-07-26T19:57:15+02:00",
"vorgangsbezug": [
{
"id": "100154",
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113

```
Titel 656 03 - Beteiligung des Bundes in knappschaftlicher
Rentenversicherung - (G-SIG: 14000059)",
"vorgangstyp": "Über- und außerplanmäßige Haushaltsausgaben"
}
],
"urheber": [
{
"einbringer": false,
"bezeichnung": "BMF",
"titel": "Bundesministerium der Finanzen"
}
],
"fundstelle": {
"pdf_url": "https://dserver.bundestag.de/brd/1998/D908+98.pdf",
"id": "82891",
"dokumentnummer": "908/98",
"datum": "1998-11-13",
"dokumentart": "Drucksache",
"drucksachetyp": "Unterrichtung",
"herausgeber": "BR",
"urheber": [
"Bundesministerium der Finanzen"
]
},
"text": "[NoTextAvailable]",
"titel": "Haushaltsführung 1998 Überplanmäßige Ausgabe bei Kapitel 1113
Titel 656 03 - Beteiligung des Bundes in knappschaftlicher
Rentenversicherung -",
"datum": "1998-11-13"
}
```


This confirms exactly what the coverage gap predicted, and it's a useful finding for your pipeline design:

What you learned:

- **The relationship is confirmed as documented** — `/drucksache-text/{id}` returns everything `/drucksache/{id}` does, plus the `text` field. Same `id`, identical metadata, no drift between endpoints. Schema assumption validated.

- **`text` isn't absent — it's the literal string `"[NoTextAvailable]"`.** This is important and easy to miss: it's not a missing key, not `null`, not an empty string. It's a **sentinel value** the API returns when full text isn't indexed (consistent with the 2003 cutoff for BR full-text search you found

earlier — this 1998 BR document predates it). Any downstream code checking `if doc.get("text")` would treat this as "present" since it's a truthy non-empty string — a real bug waiting to happen if unhandled.

This needs to be handled explicitly, in two places:

In dbt staging — don't let `"[NoTextAvailable]"` silently sit in a `text` column as if it were content:

```
-- stg_drucksache_text.sql
select
    id,
    dokumentnummer,
    titel,
    case
        when text = '[NoTextAvailable]' then null
        else text
    end as full_text,
    text = '[NoTextAvailable]' as text_unavailable_flag
from {{ source('raw', 'drucksache_text') }}
```

In your fetch decision — given this sentinel exists, and given full-text pulls are already capped at 10/page (expensive relative to metadata's 100/page), it's worth checking now: for your pension keyword set, how many of the matched Drucksachen are likely to hit this sentinel because they're pre-2003 BR documents or otherwise unindexed? A quick check:

```
# how many linked Drucksachen predate 2003 (BR) — candidates for
[NoTextAvailable]
cur = conn.execute("""
    SELECT COUNT(*) FROM drucksache
    WHERE herausgeber = 'BR' AND datum < '2003-01-01'
""")
print("BR docs pre-2003 (likely no full text):",
cur.fetchone()[0])
```

Decision point: given this sentinel behavior and the earlier finding that this particular record is a false-positive/noise match anyway, is `/drucksache-text` worth fetching at all for your early-warning use case? An early-warning system is about *detecting that something changed*, not full-text analysis — `titel`, `abstract`, `deskriptor`, `beratungsstand`, and the `pdf_url` (for an analyst to click through) likely cover the alerting need without touching `-text` endpoints at all. That would let you drop `drucksache-text` from scope entirely, simplifying both the fetch layer and the dbt project by one more model — worth locking in given the time budget.

> at this stage i am just trying to pull the information and test the whole pipeline after loading the raw data into sql lite and test dbt for

transformation. i will build after testing the pipeline

That's the right sequencing — get one clean pass end-to-end (fetch → SQLite → dbt) working on a small, known-good slice before scaling up decisions like keyword tuning or wahlperiode scoping. Here's a lean checklist for that first full-pipeline test pass, given everything you've already validated:

What you've already confirmed works (don't re-test, just use it):

- `f.titel` filtering on `/vorgang` ✓
- Cursor pagination across `/vorgang`, `/vorgangsposition`, `/drucksache`, `/drucksache-text` ✓
- `Vorgang → Vorgangsposition → fundstelle → Drucksache` link path ✓
- `Drucksache → Drucksache-Text` relationship + `[NoTextAvailable]` sentinel behavior ✓
- Logging via `fetch_log` ✓

For a first raw-to-dbt pipeline test, minimal recommended scope:

```
# small deterministic slice — enough to exercise every
table/join, not the full backfill
TEST_QUERIES = [{"f.titel": "Grundrente"}]  # only 37 Vorgänge —
small, fast, real data
```

Tables to land in SQLite for the dbt test:

```
raw.vorgang             -- from QUERIES loop
raw.vorgangsposition    -- via f.vorgang for each fetched
vorgang.id
raw.drucksache           -- via fundstelle.id where dokumentart ==
"Drucksache"
raw.fetch_log            -- already working
```

You can defer `drucksache-text` from this first pass entirely — you already validated the endpoint mechanics manually, it's not required to exercise the dbt layer, and skipping it keeps the test slice smaller and faster to iterate on.

Minimal dbt project structure to test against this raw data:

```
dbt_project/
├── models/
│   ├── staging/
│   │   ├── stg_vorgang.sql          -- select + light cleanup
from raw.vorgang
│   │   ├── stg_vorgangsposition.sql
│   │   └── stg_drucksache.sql
│   └── marts/
│       └── vorgang_with_drucksache.sql   -- join test: vorgang
```

```
-> vorgangsposition -> drucksache
└── sources.yml                          -- declare raw.vorgang etc.
as dbt sources
```

First things to verify once `dbt run` completes:

- `dbt run` succeeds with no errors — confirms `profiles.yml` correctly points at your SQLite file and the adapter reads it.
- Row counts sanity check: `stg_vorgang` row count should roughly match your `fetch_log` numbers for `Grundrente` (minus any dedup).
- The join in `vorgang_with_drucksache.sql` actually returns rows — confirms the FK relationship (`vorgangsposition.vorgang_id` → `vorgang.id`, `vorgangsposition.fundstelle.id` → `drucksache.id`) survived the JSON → SQLite flattening correctly.
- `dbt test` (even just built-in `not_null`/`unique` tests on `id` columns) — cheap to add now, catches the duplicate-row question you flagged earlier for real.

Once this small slice runs clean end-to-end, scaling to the full 5-keyword set is just re-pointing `QUERIES` — no pipeline logic changes needed.

Want to work through the `sources.yml` + `stg_vorgang.sql` skeleton next, or are you set to build that part yourself and want to check back in once `dbt run` executes?

provide the code so i can check for for dbt run and dbt test

Here's a working dbt project skeleton for the SQLite test pass. One assumption I'm making explicit: this assumes your `fetch_and_store` wrote **flattened columns** into SQLite (matching the field names you've seen in the JSON, e.g. `id`, `titel`, `datum`, `beratungsstand` as actual columns) rather than storing the whole record as a single JSON blob column. If it's the latter, flag that and I'll rewrite the staging models using `json_extract()` instead.

`dbt_project/profiles.yml`

```
dip_monitoring:
  target: dev
  outputs:
    dev:
      type: sqlite
      threads: 1
      database: "database"          # dbt-sqlite quirk: logical
name, not a path
      schema: "main"
      schemas_and_paths:
        main: "/absolute/path/to/data/raw.sqlite"   # TODO: fix
```

```
path
      schema_directory: "/absolute/path/to/data"
```

Requires `pip install dbt-sqlite`. Adjust the path — dbt-sqlite needs an absolute path here.

`dbt_project/dbt_project.yml`

```
name: 'dip_monitoring'
version: '1.0.0'
config-version: 2
profile: 'dip_monitoring'

model-paths: ["models"]

models:
  dip_monitoring:
    staging:
      +materialized: view
    marts:
      +materialized: table
```

`dbt_project/models/staging/sources.yml`

```
version: 2

sources:
  - name: raw
    schema: main
    tables:
      - name: vorgang
      - name: vorgangsposition
      - name: drucksache
      - name: fetch_log
```

`dbt_project/models/staging/stg_vorgang.sql`

```
-- One row per Vorgang as fetched. Dedup by id, keep latest by
aktualisiert
-- in case of overlapping keyword-query fetches
(Rente/Rentenversicherung/etc.)
with source as (
    select * from {{ source('raw', 'vorgang') }}
),
```

```
deduped as (
    select
        *,
        row_number() over (
            partition by id
            order by aktualisiert desc
        ) as rn
    from source
)

select
    id,
    typ,
    vorgangstyp,
    beratungsstand,
    wahlperiode,
    datum,
    aktualisiert,
    titel,
    abstract
    -- deskriptor, sachgebiet, initiative are arrays in the
source JSON —
    -- confirm how fetch_and_store persisted them (separate
table? JSON text column?)
    -- before referencing here.
from deduped
where rn = 1
```

dbt_project/models/staging/stg_vorgangsposition.sql

```
-- One row per Vorgangsposition. Carries the
Drucksache/Plenarprotokoll link
-- via fundstelle.id / fundstelle.dokumentart, flattened at fetch
time.
with source as (
    select * from {{ source('raw', 'vorgangsposition') }}
),

deduped as (
    select
        *,
        row_number() over (
            partition by id
            order by aktualisiert desc
```

```
        ) as rn
    from source
)

select
    id,
    vorgang_id,
    vorgangsposition,
    vorgangstyp,
    zuordnung,
    gang,
    fortsetzung,
    nachtrag,
    dokumentart,
    datum,
    aktualisiert,
    -- TODO: confirm actual column names for the flattened
fundstelle fields —
    -- e.g. fundstelle_id, fundstelle_dokumentart,
fundstelle_dokumentnummer
    fundstelle_id,
    fundstelle_dokumentart,
    fundstelle_dokumentnummer,
    fundstelle_pdf_url
from deduped
where rn = 1
```

dbt_project/models/staging/stg_drucksache.sql

```
with source as (
    select * from {{ source('raw', 'drucksache') }}
),

deduped as (
    select
        *,
        row_number() over (
            partition by id
            order by aktualisiert desc
        ) as rn
    from source
)

select
```

```
    id,
    typ,
    dokumentart,
    drucksachetyp,
    dokumentnummer,
    wahlperiode,
    herausgeber,
    datum,
    aktualisiert,
    titel
from deduped
where rn = 1
```

`dbt_project/models/marts/vorgang_with_drucksache.sql`

```
-- Test join: confirms Vorgang -> Vorgangsposition -> Drucksache
link
-- survived the JSON -> SQLite -> dbt round trip.
select
    v.id            as vorgang_id,
    v.titel         as vorgang_titel,
    v.vorgangstyp,
    v.beratungsstand,
    v.wahlperiode,
    vp.id           as vorgangsposition_id,
    vp.vorgangsposition,
    vp.gang,
    d.id            as drucksache_id,
    d.dokumentnummer,
    d.drucksachetyp,
    d.titel         as drucksache_titel
from {{ ref('stg_vorgang') }} v
inner join {{ ref('stg_vorgangsposition') }} vp
    on v.id = vp.vorgang_id
inner join {{ ref('stg_drucksache') }} d
    on vp.fundstelle_id = d.id
   and vp.fundstelle_dokumentart = 'Drucksache'
```

`dbt_project/models/marts/schema.yml` (tests)

```
version: 2

models:
  - name: stg_vorgang
```

```
    columns:
      - name: id
        tests:
          - unique
          - not_null

  - name: stg_vorgangsposition
    columns:
      - name: id
        tests:
          - unique
          - not_null
      - name: vorgang_id
        tests:
          - not_null

  - name: stg_drucksache
    columns:
      - name: id
        tests:
          - unique
          - not_null

  - name: vorgang_with_drucksache
    columns:
      - name: vorgang_id
        tests:
          - not_null
      - name: drucksache_id
        tests:
          - not_null
```

Run it

```
cd dbt_project
dbt debug          # confirm the sqlite connection resolves first
dbt run
dbt test
```

Before running: the `fundstelle_*` and array-field (`deskriptor`, `sachgebiet`, `initiative`) column names in the staging models are placeholders — I don't know how `fetch_and_store` actually flattened those nested/array JSON structures into SQLite columns. Can you paste the SQLite table schema (`PRAGMA table_info(vorgang);` and `PRAGMA`

`table_info(vorgangsposition);`) so I can align the column names exactly rather than guessing?

> i only worked on the code to just test dbt run and dbt test functionality , it worked and i want to test the data trasnfromed from db

Good — now that `dbt run` and `dbt test` both pass structurally, the next step is checking whether the transformed data actually *looks right*, not just that it ran without errors. Passing tests only confirms `id` uniqueness and join non-nullness — it says nothing about whether the content is sensible.

1. Check where dbt actually materialized the models

Since `staging` is set to `view` and `marts` to `table`, confirm they landed where expected:

```
import sqlite3
conn = sqlite3.connect("/absolute/path/to/data/raw.sqlite")

cur = conn.execute("SELECT name, type FROM sqlite_master WHERE
type IN ('table','view') ORDER BY name")
for row in cur.fetchall():
    print(row)
```

You should see `stg_vorgang`, `stg_vorgangsposition`, `stg_drucksache` as views, and `vorgang_with_drucksache` as a table.

2. Row count sanity check — raw vs. transformed

```
import pandas as pd

for table in ["vorgang", "vorgangsposition", "drucksache"]:
    raw_count = pd.read_sql(f"SELECT COUNT(*) as n FROM {table}",
conn)["n"][0]
    stg_count = pd.read_sql(f"SELECT COUNT(*) as n FROM
stg_{table}", conn)["n"][0]
    print(f"{table}: raw={raw_count}, staged={stg_count}")
```

If `staged < raw`, that's your dedup logic (the `row_number() ... where rn = 1`) actually removing duplicate `id`s — worth confirming that's the expected amount of shrinkage rather than something unexpected being dropped.

3. Eyeball the actual transformed rows

```
df_vorgang = pd.read_sql("SELECT * FROM stg_vorgang LIMIT 10",
conn)
df_vorgang
```

```
df_joined = pd.read_sql("SELECT * FROM vorgang_with_drucksache
LIMIT 10", conn)
df_joined
```

Specifically check:

- Does `stg_vorgang` still contain the 1998 knappschaftliche-Rentenversicherung false-positive record, or whatever slice you actually loaded? Confirms you're looking at the data you think you're looking at.
- In `vorgang_with_drucksache`, does `vorgang_titel` and `drucksache_titel` line up sensibly for a few rows — i.e., is the join matching the *correct* Drucksache to the *correct* Vorgang, not just matching *some* row due to a key collision?

4. Compare join coverage — how many Vorgänge got dropped by the inner joins?

```
total_vorgang = pd.read_sql("SELECT COUNT(*) as n FROM
stg_vorgang", conn)["n"][0]
joined_vorgang = pd.read_sql("SELECT COUNT(DISTINCT vorgang_id)
as n FROM vorgang_with_drucksache", conn)["n"][0]
print(f"Vorgänge with a resolved Drucksache link:
{joined_vorgang} / {total_vorgang}")
```

Since your mart uses `inner join` twice, any Vorgang whose `Vorgangsposition` links only to a `Plenarprotokoll` (not a `Drucksache`) — or has no `Vorgangsposition` at all — silently disappears from this mart. That's expected behavior given the `and vp.fundstelle_dokumentart = 'Drucksache'` filter, but worth confirming the drop-off rate is what you'd expect rather than near-100%, which would suggest something's broken in the join keys instead.

Run these and share what comes back — particularly the raw-vs-staged counts and the join coverage ratio — and I can help you spot whether the numbers look like clean data or a symptom of something in the flattening/dedup logic.

OperationalError Traceback (most recent call last) Cell In[12], line 2 1 import sqlite3 ----> 2 conn = sqlite3.connect("/absolute/path/to/data/raw.sqlite") 3 4 cur = conn.execute("SELECT name, type FROM sqlite_master WHERE type IN ('table','view') ORDER BY name") 5 for row in cur.fetchall(): OperationalError: unable to open database file

That error's on me — `/absolute/path/to/data/raw.sqlite` was a literal placeholder in my example, not a real path. You need to point it at the actual SQLite file your notebook has been writing to all along.

Find the real path — you already have an open connection (`conn`) in your notebook from the fetch steps. Check what it's pointing at:

```
# if you still have your original notebook connection object
alive, check this instead:
```

```
cur = conn.execute("PRAGMA database_list")
print(cur.fetchall())
```

That returns something like `[(0, 'main', '/home/you/dip-monitoring/data/raw.sqlite')]` — the third element is your real path.

If that connection isn't in scope anymore, find the file directly:

```
import subprocess
print(subprocess.run(["find", ".", "-name", "*.sqlite"],
capture_output=True, text=True,
cwd="/path/to/your/project/root").stdout)
```

Or just check wherever your `db.py` / `create_connection`-type function set the path — likely something like `data/raw.sqlite` relative to your project root, which is why the literal `/absolute/path/to/...` failed.

Once you have the real path, two things to verify:

- Reconnect with the correct path:

```
conn = sqlite3.connect("<your-actual-path>/raw.sqlite")
```

- Confirm dbt was actually pointed at the **same file** — this is the more important check. Open `dbt_project/profiles.yml` and look at what path you put in `schemas_and_paths`. If that path doesn't match where your fetch script has been writing raw data, `dbt run` would have succeeded (it can create/materialize views against an empty or wrong database without erroring) but produced views over the wrong — or empty — source tables. Worth confirming `dbt debug` output and this reconnect point at the identical file before trusting any row counts.

What path did you actually use in `profiles.yml`, and does it match where your raw fetch data lives?

i ran below commend on the terminal and it works sqlite3 ..\dip.sqlite "SELECT titel, beratungsstand, aktualisiert FROM mart_rentenpolitik_monitor LIMIT 10;"

Good — that confirms two things: the real path is `../dip.sqlite` (relative to wherever your terminal's cwd was, likely your dbt project folder), and you've already built a mart called `mart_rentenpolitik_monitor` — beyond the `vorgang_with_drucksache` skeleton I sketched, so you've clearly extended the models yourself. That's fine — let's just verify what's actually in it.

1. Reconnect from your notebook with the correct relative path

Path resolution from a notebook depends on its working directory, so best to be explicit:

```
import sqlite3
```

```
from pathlib import Path

# adjust based on where your notebook actually lives relative to
dip.sqlite
db_path = Path("../dip.sqlite").resolve()
print(db_path, db_path.exists())  # sanity check before
connecting

conn = sqlite3.connect(db_path)
```

2. See what tables/views actually exist now

```
cur = conn.execute("SELECT name, type FROM sqlite_master WHERE
type IN ('table','view') ORDER BY type, name")
for row in cur.fetchall():
    print(row)
```

This tells us the real model list — useful since `mart_rentenpolitik_monitor` wasn't in what I sketched, so I don't know its schema or what joins/filters it applies.

3. Pull it into pandas for a proper look, not just the 3 columns from your terminal check

```
import pandas as pd
df = pd.read_sql("SELECT * FROM mart_rentenpolitik_monitor",
conn)
print(df.shape)
df.head(10)
```

4. Now the actual data-quality checks worth running against this mart:

```
# any nulls in fields that should always be populated?
print(df.isnull().sum())

# distribution of beratungsstand — gives a sense of how many are
"open" vs "closed"
print(df["beratungsstand"].value_counts())

# distribution of wahlperiode, if present — confirms historical
spread
if "wahlperiode" in df.columns:
    print(df["wahlperiode"].value_counts().sort_index())

# duplicate check on whatever the natural key is (likely vorgang
id)
id_col = "id" if "id" in df.columns else "vorgang_id"
```

```
print("rows:", len(df), "distinct ids:", df[id_col].nunique())
```

Since I don't have the SQL behind `mart_rentenpolitik_monitor`, could you paste that model's `.sql` file? That'll let me check whether the filtering/join logic matches what we discussed (e.g. whether it's pulling in the `Abgeschlossen`-from-1998 style false positives, whether it dedupes correctly, and whether the `Drucksache` link resolves the way we traced manually) — rather than me guessing at what it should contain.

C:\Users\padma\Desktop\CaseStudy\dip.sqlite

Got it — that's a Windows path, so watch backslash escaping in Python (use a raw string or forward slashes, or `pathlib`).

```
import sqlite3
from pathlib import Path

db_path = Path(r"C:\Users\padma\Desktop\CaseStudy\dip.sqlite")
print(db_path.exists())  # confirm before connecting

conn = sqlite3.connect(db_path)
```

Then run the discovery + quality checks:

```
# what tables/views exist
cur = conn.execute("SELECT name, type FROM sqlite_master WHERE
type IN ('table','view') ORDER BY type, name")
for row in cur.fetchall():
    print(row)
```

```
import pandas as pd
df = pd.read_sql("SELECT * FROM mart_rentenpolitik_monitor",
conn)
print(df.shape)
df.head(10)
```

```
print(df.isnull().sum())
print(df["beratungsstand"].value_counts())
if "wahlperiode" in df.columns:
    print(df["wahlperiode"].value_counts().sort_index())

id_col = "id" if "id" in df.columns else "vorgang_id"
print("rows:", len(df), "distinct ids:", df[id_col].nunique())
```

Once that runs, paste the output (shape, null counts, `beratungsstand` distribution, row-vs-distinct-id counts) and — if you can — the `mart_rentenpolitik_monitor.sql` model itself, so I can check the transformation logic against what we traced manually (the false-positive risk from `f.titel` substring matching, the `Abgeschlossen`/historical noise, and

whether the Drucksache join is inner or left).

```
vorgang_id 0
titel 0
vorgangstyp 0
beratungsstand 70
wahlperiode 0
datum 0
aktualisiert 0
initiativen 1078
sachgebiete 1197
abstract 205
gesta 1663
dip_url 0
last_ingested_at 0
dtype: int64
beratungsstand
Beantwortet 1209
Abgeschlossen - Ergebnis siehe Vorgangsablauf 174
Abgelehnt 118
Verkündet 62
Nicht abgeschlossen - Einzelheiten siehe Vorgangsablauf 51
Erledigt durch Ablauf der Wahlperiode 26
Für erledigt erklärt 15
Angenommen 14
Abgeschlossen 8
Zusammengeführt mit... (siehe Vorgangsablauf) 7
Noch nicht beraten 4
Überwiesen 3
Zurückgezogen 3
Beschlussempfehlung liegt vor 2
In der Beratung (Einzelheiten siehe Vorgangsablauf) 1
(1767, 13)

Name: count, dtype: int64
wahlperiode
7 9
8 137
9 56
10 99
11 108
12 87
13 106
```

```
14 119
15 116
16 150
17 191
18 168
19 208
20 135
21 78
Name: count, dtype: int64
rows: 1767 distinct ids: 1767
```

This is genuinely good data — worth walking through what it's telling you before deciding next steps.

The good news first:

- `rows: 1767, distinct ids: 1767` → **no duplicates.** Your dedup logic (or the mart's grouping) is working correctly across the overlapping keyword queries. This was the exact risk we flagged early on, and it's clean.
- Row 7 (`vorgang_id=337670`) is your originally-traced smoke-test record — "Abschaffung der abschlagsfreien Rente" — present and consistent. Good continuity check.
- The `beratungsstand` distribution is genuinely useful: 1,209 `Beantwortet` (answered written questions), and a long tail of closed/historical statuses (`Abgeschlossen`, `Erledigt durch Ablauf der Wahlperiode`, etc.) mixed with a small number of *currently open* items (`Noch nicht beraten`: 4, `Überwiesen`: 3, `In der Beratung`: 1, `Beschlussempfehlung liegt vor`: 2). **That handful is your actual early-warning signal** — the rest is context/history.

Now the things worth flagging:

- **This is your full historical pull, not scoped data** — the `wahlperiode` distribution confirms it spans WP 7 (1976!) through WP 21 (current), 15 election periods. That's expected given you deliberately kept `f.wahlperiode` commented out for the raw layer. But it confirms the decision we discussed earlier is now concrete: **this mart as-is is not an early-warning feed**, it's a full historical archive. You'll want a filtered view/mart on top of this — e.g. `wahlperiode = 21 AND beratungsstand NOT IN ('Abgeschlossen...', 'Erledigt...', 'Verkündet', ...)` — before this becomes something an analyst actually monitors day-to-day.

- **70 nulls in `beratungsstand`** — worth a quick look at which 70 rows these are:

```
df[df["beratungsstand"].isnull()][["vorgang_id", "titel",
"vorgangstyp", "wahlperiode"]].head(10)
```

Is this a real API gap (some Vorgänge genuinely have no `beratungsstand`), or a transformation issue (e.g. a join/cast dropping values for a specific `vorgangstyp`)? If it correlates with one `vorgangstyp` in particular, that's a clue.

- **High nulls on `initiativen` (1078), `sachgebiete` (1197), `gesta` (1663)** — these are mostly expected. Recall: `Schriftliche Frage` (written questions) — your dominant type at 1,209 rows — typically won't have `initiative` (that's for bills) or `gesta` (legislative tracking numbers, bill-specific) populated. Worth confirming the null rate roughly matches the `vorgangstyp` split (i.e., nulls cluster in `Schriftliche Frage` rows, not randomly across bill-type rows) — that would confirm it's real API sparsity, not a broken transform:

```
df.groupby("vorgangstyp")[["initiativen", "sachgebiete",
"gesta"]].apply(lambda x: x.isnull().mean())
```

- **`abstract` has HTML in it** — row 8/9 show `Originaltext der Frage(n):<br />\r\n<br />\r\n...`. If this field is ever surfaced to an end user (e.g. in an alert email/dashboard), you'll want to strip HTML tags at some point in the transform chain — not urgent for pipeline testing, but worth a note for later.

Bottom line: the pipeline is working correctly end-to-end — fetch → SQLite → dbt → clean, deduplicated, joined output. What you're looking at now is a full historical archive rather than a live monitoring feed, which was always the plan for the raw/staging layer. The next real decision is building the filtered "current + open" mart on top of this that would actually serve as your early-warning view — want to sketch that model next?

Good — let's build the filtered "active/early-warning" mart on top of what you've already validated. Given the messy, inconsistent `beratungsstand` strings you just surfaced (15 distinct values, mixing genuinely open states like `"Noch nicht beraten"` with closed ones like `"Erledigt durch Ablauf der Wahlperiode"`), a hardcoded `WHERE beratungsstand NOT IN (...)` list buried in SQL is going to be hard to maintain and easy to get subtly wrong. A **dbt seed** (a CSV you control, joined in) is more transparent and easier to troubleshoot — exactly your stated goal.

`dbt_project/seeds/dip_status_mapping.csv`

```
beratungsstand,status_category
Beantwortet,closed
```

```
Abgeschlossen - Ergebnis siehe Vorgangsablauf,closed
Abgelehnt,closed
Verkündet,closed
Nicht abgeschlossen - Einzelheiten siehe Vorgangsablauf,open
Erledigt durch Ablauf der Wahlperiode,closed
Für erledigt erklärt,closed
Angenommen,closed
Abgeschlossen,closed
Zusammengeführt mit... (siehe Vorgangsablauf),closed
Noch nicht beraten,open
Überwiesen,open
Zurückgezogen,closed
Beschlussempfehlung liegt vor,open
In der Beratung (Einzelheiten siehe Vorgangsablauf),open
```

This is your judgment call to review, not mine — I mapped based on the German phrasing (e.g. "Noch nicht beraten" = not yet debated = open; "Erledigt" = done = closed), but you should sanity-check each line against what a Public Affairs analyst would actually want flagged as "still moving."

`dbt_project/dbt_project.yml` — add a var for current Wahlperiode

```
vars:
  current_wahlperiode: 21
```

Keeping this as a var (not hardcoded in the SQL) means next election period you change one line, not every model.

`dbt_project/models/marts/mart_rentenpolitik_alerts.sql`

```
-- Early-warning view: current Wahlperiode + not-yet-closed
Vorgänge only.
-- Sorted by most recently updated first — this IS the monitoring
feed.

with base as (
    select * from {{ ref('mart_rentenpolitik_monitor') }}
),

status_mapped as (
    select
        b.*,
        coalesce(s.status_category, 'unknown') as status_category
    from base b
```

```
    left join {{ ref('dip_status_mapping') }} s
        on b.beratungsstand = s.beratungsstand
)

select
    vorgang_id,
    titel,
    vorgangstyp,
    beratungsstand,
    status_category,
    wahlperiode,
    datum,
    aktualisiert,
    initiativen,
    sachgebiete,
    abstract,
    dip_url,
    last_ingested_at,
    -- flag items updated in the last 14 days as "new activity"
for the alert feed
    julianday('now') - julianday(aktualisiert) <= 14 as
is_recent_update
from status_mapped
where wahlperiode = {{ var('current_wahlperiode') }}
  and status_category in ('open', 'unknown')  -- surface
'unknown' too, don't silently drop unmapped statuses
order by aktualisiert desc
```

Why `status_category in ('open', 'unknown')` rather than just `'open'`: if a new `beratungsstand` string appears in a future fetch that isn't yet in your seed CSV, `coalesce` turns it into `'unknown'` rather than silently `NULL`-filtering it out of the join. Surfacing `unknown` rows in the alert feed (rather than dropping them) means you'll *notice* an unmapped status and can add it to the seed — instead of quietly missing a real Vorgang because your mapping was incomplete. This is a deliberate fail-safe/fail-loud choice worth keeping given this is an early-warning system.

`dbt_project/models/marts/schema.yml` — add tests

```
version: 2

models:
  - name: mart_rentenpolitik_alerts
    columns:
```

```
      - name: vorgang_id
        tests:
          - unique
          - not_null
      - name: status_category
        tests:
          - not_null
          - accepted_values:
              values: ['open', 'unknown']   # 'closed' should
never appear here — tests the filter itself

seeds:
  - name: dip_status_mapping
    columns:
      - name: beratungsstand
        tests:
          - unique
          - not_null
      - name: status_category
        tests:
          - not_null
          - accepted_values:
              values: ['open', 'closed']
```

Run it

```
dbt seed        # loads dip_status_mapping.csv into the db first
dbt run
dbt test
```

Once that's clean, pull it into pandas the same way as before:

```
df_alerts = pd.read_sql("SELECT * FROM
mart_rentenpolitik_alerts", conn)
print(df_alerts.shape)
df_alerts[["titel", "beratungsstand", "status_category",
"aktualisiert", "is_recent_update"]]
```

Given your earlier row-level check, expect this to shrink dramatically from 1,767 down to roughly single-to-low-double digits (the `Noch nicht beraten`: 4, `Überwiesen`: 3, `In der Beratung`: 1, `Beschlussempfehlung liegt vor`: 2 rows, all at WP 21, plus any `unknown` stragglers) — that small, high-signal list is exactly what an early-warning feed should look like. Run it and let me know what comes back.

donwload the chat history so far

This is a reconstructed summary of our conversation (not a verbatim export, since I don't have a raw-transcript-download function) — organized by the pipeline stages we worked through: API discovery, schema exploration, keyword/entity decisions, manual relationship tracing, pipeline build/test, and the early-warning mart design, including all the code snippets and key findings (like the `f.titel` undocumented parameter and the `[NoTextAvailable]` sentinel). Should be usable as reference material or a basis for your case study write-up.

Dip pension pipeline chat history
MD · Click to download

> i do not need summary i want the whole conversation to be downloaded

Understood — here's the full conversation, turn by turn, as close to verbatim as I have access to (I don't have a raw transcript-export tool, so this is reconstructed from what's in this conversation).

One caveat worth flagging: there's no built-in "export chat" function available to me, so this is a manual, turn-by-turn reconstruction from what's visible in our conversation — not an automated system export. It should match closely (all code blocks, pasted outputs, and full responses are included), but if you need a guaranteed-exact copy, claude.ai also lets you export conversations from account settings, which would be the authoritative source.

Full conversation transcript
MD · Click to download